DSPB Group Project
Fire Insurance Premium Prediction

Agricultural Portfolio · Crédit Agricole Assurances
Data Science Projects for Business · March 2026
Alizée Archereau

The report and notebooks follow the data science lifecycle: **Phase 1** covers **data cleaning and preprocessing**, **Phase 2** applies **unsupervised learning** (PCA and clustering) to explore the structure of the portfolio, and **Phase 3** builds the **supervised prediction model** for FREQ, CM, and the final CHARGE.

Note that the Numbering of subparts is close to but not exactly like the Numbering in the Notebook.

Table of Contents

Phase 1 — Data Preprocessing & Cleaning

1.1 Data Overview & EDA

The training set contains **166,943 contracts** with **373 features** and three targets: `FREQ` (annual claim frequency), `CM` (average cost per claim), and `CHARGE = FREQ × CM × ANNEE_ASSURANCE` (the final premium).

The first thing we noticed during EDA was **mixed data types** throughout the dataset. Many columns stored numbers as strings: range bins like `"21-30"`, text codes like `"ANNEE5"`, and censored values like `"7000+"` for top-coded surface areas. We extracted a sample to a spreadsheet to catalogue every format before writing any cleaning code. This let us define explicit targeted conversions for each column group rather than applying a risky general rule.

The second major issue was **block-missing data**. A group of columns was entirely missing for the same rows simultaneously (mean pairwise missingness correlation ≈ 1.0). This meant a **MNAR** (Missing Not At Random) pattern, where the absence of data is informative in itself. Our hypothesis is that these contracts were entered in an older "legacy" IT system that did not record those fields. We went ahead with this hypothesis based online research on similar-looking missing data patterns as well as previous experience in Big Data position with legacy systems (in SAP), which provides a "good enough" domain comprehension for this assignment. *(See Appendices)*

On a secondary note, six contracts in the training labels had negative CHARGE values. The cause is unknown (possible refunds or corrections from a prior period). These were clipped to zero before training and evaluation.

1.2 Handling MNAR — Pattern Mixture Approximation

Following the provided decision tree for missing data decisions: MNAR + domain knowledge available + systematic pattern → **Pattern Mixture Models**. However, a full pattern mixture model estimates a separate distribution per missing pattern, which is expensive at 373 features. We decided to use a lightweight approximation instead:

- **Extract the pattern** into a single binary flag (`meta_missing_legacy_data = 1` for legacy contracts). The missingness pattern itself is the signal.
- **Drop the sparse columns** above a phase-specific threshold (see below). The flag can compensate for the lost information if `drop_threshold < flag_threshold` (see the PCA pipeline) ; otherwise, it just gives the model access to that domain information.
- The flag is computed on columns **missing between 56% and 59%,** capturing exactly the legacy block. Columns above the phase-specific drop threshold are *then* removed, but the flag remains.

1.3 Cleaning Pipeline (`_core_clean`)

`_core_clean` is a single shared function used by both phases 2 and 3. Steps in order:

- **NaN standardisation:** all placeholder strings (`'NC'`, `'ND'`, `'?'`, `'i'`, `''`) replaced with `np.nan`.

- **Top-coding:** '7000+' in surface columns replaced with 7500 (midpoint), plus a binary _is_topcoded flag so the model knows the value was censored.
- **Targeted ordinal extraction:** only known bin-column prefixes (DISTANCE_*, ALTITUDE_*, PROPORTION_*, MEN_COLL) parsed to their lower bound via str.extract(r'^(\d+)'). A broad automatic regex initially used was ruled out as it messed up unrelated columns that happened to start with digits.
- **Explicit binary maps:** O/N flags (ADOSS, INDEM1, DEROG1-16, KAPITAL34-43) and 1/2 flags (TYPERS, CARACT2) mapped to 0/1. All columns cast to object first to avoid a pandas category dtype error.
- **Column-specific fixes:** FRCH2 ('A' → 99), CARACT4 (text ranges → ordinal 0–5), COEFASS (date-range strings → integers).
- **Category dtype:** remaining string columns cast to pandas.Categorical for XGBoost native handling (no label encoding needed).

Why different thresholds between phases? Phase 2 (Unsupervised Learning, ie PCA and Clustering) uses a strict **50% drop threshold**: PCA requires a well-filled covariance matrix and is sensitive to sparse columns. Phase 3 (Supervised Learning ie our Prediction model) uses a loose **95% threshold**: At each split, XGBoost tries both directions for missing values and keeps whichever reduces the loss more, so keeping sparse columns adds signal at no cost.

Phase 2 — Unsupervised Learning

2.1 Preprocessing Pipeline for PCA

All columns are imputed, encoded, and scaled before PCA. The pipeline groups columns by missingness rate (as suggested by the Missing data handling guidelines) and data type:

Column group	Numeric treatment	Categorical treatment
0% missing	StandardScaler	FrequencyEncoder → StandardScaler
0–15% missing	Median impute → StandardScaler	Mode impute → FrequencyEncoder → StandardScaler
>15% missing	Median impute → StandardScaler	Mode impute → FrequencyEncoder → StandardScaler

FrequencyEncoder replaces each category with its proportion in the training set: this is basically a smooth, bounded numeric encoding. One-hot encoding was ruled out because it would add thousands of sparse binary columns and make PCA components uninterpretable (we tried..). Note: the category dtype had to be converted back to object before this pipeline runs, as sklearn's SimpleImputer does not support it. The category dtype is only needed for XGBoost in Phase 3.

2.2 PCA

PCA was run on the fully preprocessed matrix. Around **80 components** are needed to explain 80% of the variance *(See Appendices)*, confirming the data is genuinely high-dimensional with many independent sources of variance. The first six PCs each align with distinct business concepts and offer interesting insights:

PC	% Var	Main drivers	Business interpretation
PC1	9.2%	SURFACE*, CAPITAL*	Physical scale of the farm
PC2	4.3%	RISK1–RISK5	Actuarial risk profile
PC3	3.8%	ALTITUDE_*	Topography and climate zone
PC4	3.1%	MEN_*, LOG_SOC	Socio-demographic profile
PC5	2.4%	NBBAT*	Building stock
PC6	2.1%	DEROG*	Pricing exceptions

The 373 features are not redundant noise, they reflect genuinely different dimensions of farm fire risk. The way to convey those information to non-technical personnel would be: “A farm's fire risk is not one-dimensional. The PCA tells us it breaks down into at least six independent axes: how big the farm is, how risky it is rated, where it is geographically, who owns it, and how many buildings are on it. A large farm in a high-altitude zone with many buildings is a different risk from a small farm in a flat area, and the model needs all six dimensions to price them correctly.”

2.3 Clustering

K-Means (k=4) was applied on the 80%-variance PCA “subspace”. Using all 373 raw features would let noise dominate the distance calculation between farms.

Cluster	FREQ	Avg CM (€)	Business profile
0	0.024	425	Moderate frequency, high severity. Large, high-value farms.
1	0.007	81	Low risk. Bulk of the portfolio.
2	0.008	93	Low risk. Similar to cluster 1.
3	0.057	763	Highest risk: most frequent AND most expensive.

(See Appendices)

The key insight from the heatmap is that **high FREQ and high CM do not always coincide**. Cluster 3 is the exception: both the highest claim frequency (0.057) and the highest average cost per claim (763€). In PCA space it sits at high PC1 values -> the largest farms physically. More buildings, more capital, more ignition sources.

The way to convey this to non-technical personnel would be: "Most farms are low-risk and cheap to insure. But a small group of large farms (the ones with the most buildings and the most capital at risk) catches fire more often and costs more when it does. Charging everyone the same premium would mean the safe farms subsidise the dangerous ones."

Clusters 1 and 2 account for the bulk of contracts but contribute little to total losses. A flat pricing approach would massively underprice cluster 3. This finding also motivates the two-part model in Phase 3: frequency and severity must be modelled independently because they are driven by different features and follow different statistical distributions.

Phase 3 — Supervised Learning (Prediction)

3.1 Model Rationale

Why two sub-models? (Apart from following the instructions..) To understand this we initially tried predicting CHARGE directly. The result is dominated by squared errors on rare catastrophic fires (e.g. 500k€ events that are stochastic by nature). Splitting into two sub-models gives each a better-posed problem:

$$\mathrm{CHARGE} = \mathrm{FREQ} \times \mathrm{CM} \times \mathrm{ANNEE\_ASSURANCE}$$

- model_freq -> **Poisson loss function:** adapted for count data (annual claim rate). Weighted by ANNEE_ASSURANCE so longer contracts contribute proportionally more.
- model_cm -> **Gamma loss function:** adapted for positive right-skewed costs. Trained **only on rows where CM > 0** (actual claims). Weighted by expected claim count (FREQ × ANNEE_ASSURANCE).

A flat predicted-vs-actual scatterplot *(See Appendices)* was therefore expected and logical: we predict an *expected premium* (a population mean), not the outcome of a specific random fire event.

Why track MedAE alongside RMSE? RMSE is the competition metric but is dominated by a handful of catastrophic fires. A model that predicts near-zero for everyone minimises RMSE on this zero-inflated dataset but is operationally meaningless (MedAE ≈ 0). We used MedAE as a sanity check: **RMSE for competition performance, MedAE to verify premiums are meaningful**.

3.2 Feature Engineering

Groups of related sparse columns (all SURFACE columns, all RISK scores, etc.) are aggregated into summary features. The individual sub-columns are correlated and partially sparse; their sum allow up to capture the quantities with one clean number:

- TOTAL_SURFACE, TOTAL_CAPITAL: total exposed surface, total insured value.
- CAPITAL_PER_M2: Capital per unit surface. high values mean dense, expensive assets.
- RISK_SUM, RISK_MEAN: aggregate risk score.
- PREV_SUM: total prevention measures in place.
- TOTAL_BUILDINGS: number of insured structures.

3.3 Model Iterations & Comparisons

It took use multiple tries to get to our v16 (final). Here were some landmark versions with their Out-of-fold RMSE (approx). Where tried on the platform, we also have the Test RMSE.

Version	Approach	OOF RMSE	Test RMSE	Key finding
Baseline	Predict y_train mean for all rows	8,400	5613	Lower bound. No model should do worse.

Version	Approach	OOF RMSE	Test RMSE	Key finding
v11	HistGradientBoosting + extreme L2 regularisation	6,800		MedAE ≈ 0. Predicts near-zero. Wrong.
v12	XGBoost Poisson/Gamma + CV (5-fold OOF)	6800	**5598.1**	First correct actuarial setup. Best test.
v13	v12 + tighter regularisation	6500	5613.8	Better OOF, worse test. Overfit to CV.
v14 (DL)	Residual MLP + entity embeddings (PyTorch)	6900	5604	Did not beat XGBoost on tabular data.
v16 (final)	XGBoost Poisson/Gamma + CV (5-fold OOF)+ CM and FREQ tuned (RandomizedSearchCV)	6400	5596.2	Best CV. Submitted.

v11: extreme regularisation pushed predictions toward zero — MedAE ≈ 0, which means the model priced almost every farm at zero. Useless in practice. The fix in v12 was weighting each contract by how many years it was insured (ANNEE_ASSURANCE), so the model learns a rate per year rather than a raw count.

v13 vs **v12**: stricter settings improved performance on the training folds but made it worse on the actual test set by 15 points. v12's slightly looser fit was more robust.

v14 (Deep Learning): a neural network with special handling for categorical features. Reached test RMSE 5604: close but not better than tuned XGBoost. Neural networks also need a GPU to train and are hard to explain to a regulator who asks why a specific farm was priced a certain way.

v16: both sub-models tuned via random hyperparameter search. Best overall result at 5596.2.

Note that surprisingly, predicting the mean for everyone scored 5613 on the test set, almost matching our tuned models! This shows how much a few catastrophic fires inflate the RMSE regardless of model quality.

3.4 Key Design Choices

- **Poisson / Gamma loss function:** match the statistical nature of each target. MSE on count or cost data ignores distributional structure and can produce negative predictions. These were hinted at by the challenges baseline description on the website.
- **Exposure weighting (ANNEE_ASSURANCE):** a 10-year contract has 10× more chances of producing a claim. Without this, the model learns raw counts, not annual rates.
- **Claims-only CM training:** the severity model trains on CM > 0 rows only. Including zero rows biases it toward predicting zero cost, but we want the model must learn *how much a fire costs given that one happened*, not *whether a fire will happen*.
- **5-fold OOF cross-validation:** every row is predicted by a model that has never seen it, giving an unbiased generalisation estimate across the full training set.
- **Cap at 99.9th / 99th percentile:** extreme outliers in FREQ and CM clipped during training. Prevents the loss from being dominated by a handful of catastrophic events.
- **MedAE + RMSE reporting:** RMSE for competition; MedAE to verify the model produces meaningful non-zero premiums for the typical contract.

3.5 Hyperparameter Tuning

FREQ parameters were fixed from v12 (test RMSE 5598). Re-tuning FREQ on 166k rows takes over 30 minutes and showed only marginal gains in earlier experiments.

CM was tuned with RandomizedSearchCV (30 iterations, 5-fold CV) on claim rows only (10k rows), making the search fast. The search covered: max_depth [3–5], learning_rate [0.01–0.1], n_estimators [150–500], reg_lambda [2–15], min_child_weight [10–50], subsample / colsample_bytree.

The tunings returned:

Parameter	**FREQ**	**CM**	**Motivation**
objective	count:poisson	reg:gamma	Distributional match to target type
max_depth	4	3	Shallow trees generalise better on rare events
min_child_weight	10	50	Minimum leaf volume. Avoids splits on isolated large claims
learning_rate	0.05	0.01	Step size at each boosting round
reg_lambda (L2)	3.0	15.0	Penalises large weights in high-dimensional sparse data
colsample_bytree	0.7	0.6	% of features per tree. Adds randomness, reduces variance
reg_alpha (L1)	0.2	—	Encourages feature sparsity
gamma	0.1	—	Minimum loss reduction needed to make a split

Phase 4 — Deployment & Business Context

4.1 Deployment Strategy

The scoring pipeline consists of 2 XGBoost models and a preprocessing function. We recommend two deployment models depending on the needs:

- **Real-time pricing API:** at Crédit Agricole, a broker quoting a new farm would get an instant premium estimate. The contract details are sent to the API, cleaned, scored, and FREQ, CM and CHARGE come back in milliseconds. This could plugs into any existing broker portal via a standard HTTP call.
- **Overnight batch:** every night, the full portfolio of existing contracts could be re-scored and the output fed into the renewal pricing system (the system that decides what premium to charge each existing client when their contract comes up for renewal, using the up-to-date infos on frequency etc). The pricing team could also use this run to check whether predictions are drifting over time, which would signal the model needs retraining.

Note that unlike the Phase 2 pipeline which has fitted scalers and encoders that need to be saved, `_core_clean` is just a series of rules (replace this string, map this value, extract this number). It doesn't learn anything from the data, so there is nothing extra to save or version. The only files needed in production are the two XGBoost model files.

4.2 Model Selection Justification & Explainability

Criterion	XGBoost v16	Deep Learning v14	Linear baseline
Predictive performance on Validation	Best (CV 6800)	Weaker (6900 OOF)	Worst (8400)
Explainability	SHAP values, feature-level	Black box	Full transparency
Regulatory fit	Good (SHAP justifies pricing)	?	Best
Compute cost	Minutes (CPU)	30–60 min (GPU needed)	Seconds
Missing value handling	Native routing (no imputation)	Requires full imputation	Requires imputation

In insurance pricing, explainability is very important. Regulators and clients can request justification for a premium. XGBoost with SHAP values allows accountable people to say eg: *"this premium is higher because CAPITAL_PER_M2 is elevated and RISK_SUM places this farm in the top decile"*. The deep learning model could not provide this easily, which is why we did not focus on it although it seemed promising.

To understand what drives the model's predictions, we computed **SHAP values** on a subsample (500 contracts). SHAP decomposes each prediction into the contribution of each individual feature. *(See Appendices)*

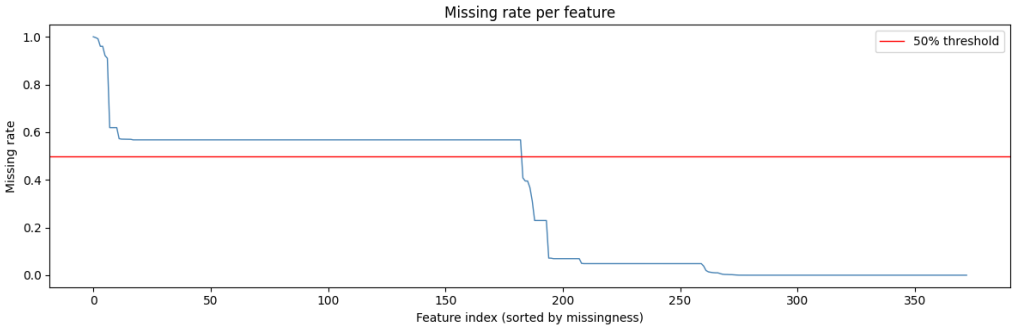
- **Globally,** the most important drivers of claim frequency are insured capital (KAPITAL32), contract duration (ANNEE_ASSURANCE), surface area, and number of structures — all physical variables, which is consistent with the clustering findings in Phase 2.
- **At the individual contract level,** SHAP produces a waterfall chart showing exactly how each feature pushes the prediction above or below the portfolio average. This is the kind of explanation a regulator or client would ask for: understanding "the premium is X because this farm has high capital and large surface areas." rather than just getting "the premium is X".

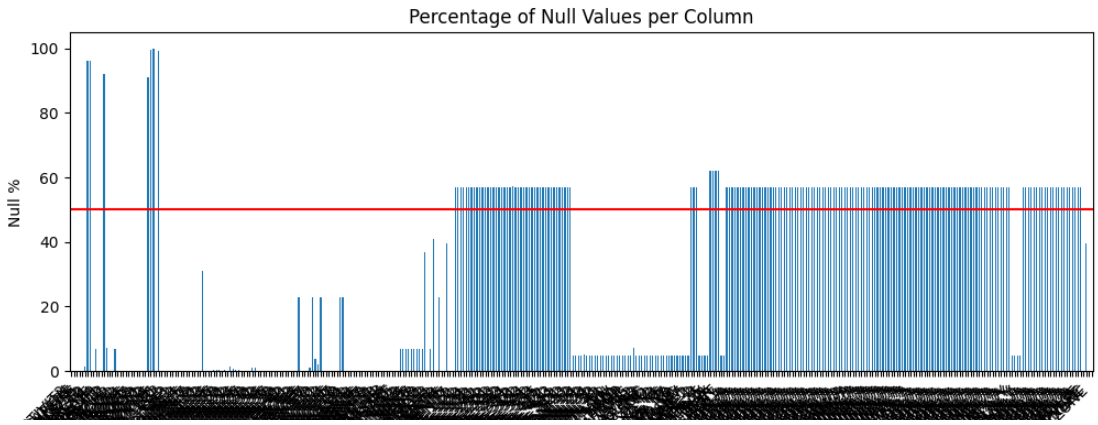
Appendices

1.1 Data Exploration

	Variable communiquée	Libellé communiqué	Data Type (Numerical	Sample Data
6	AN_EXERC	Fiscal year	Ordinal	ANNEE1, ANNEE2, ...
7	ACTIVIT2	Activity	Nominal	ACT1, ACT2, ACT, 3, ...
8	VOCATION	Purpose of the professiona	Nominal	VOC1, VOC2, VOC3, ...
9	TYPERS	Type of person	Binary Flag	1, 2
10	ANCIENNETE	Contract seniority	Numeric	0, 1, 2, ...
11	ESPINSEE	INSEE area	Nominal	ESP1, ESP2, ...
12	ZONE	Risk zone	Nominal	1, 2, 3, ...
13	CARACT1	Building characteristic	Nominal	O, N, R
14	CARACT2	Building characteristic	Binary Flag	1, 2
15	CARACT3	Building characteristic	Nominal	O, N, R
16	CARACT4	Building characteristic	Ordinal	absence de surface, Surface
17	CARACT5	Building characteristic	Numeric	0, 1, 2, ...
18	INDEM1	Compensation method 1	Binary Flag	O, N
19	TYPBAT1	Building type	Nominal	gibier-plumes, lapin, porcs,
20	INDEM2	Compensation method 2	Ordinal	CLASS1, CLASS2, ...
21	TYPBAT2	Building type	Ordinal	0, 1, 2
22	FRCH1	Deductiible level	Ordinal	0, 1, 2, ...
23	FRCH2	Deductiible level	Ordinal	1, 2, 3, ...
24	DEROG1	Pricing derogation	Binary Flag	O, N
34	DEROG11	Pricing derogation	Binary Flag	O, N
35	DEROG12	Pricing derogation	Nominal	D03, D04, D12, ...
36	DEROG13	Pricing derogation	Nominal	D03, D04, D12, ...
37	DEROG14	Pricing derogation	Nominal	D03, D04, D12, ...
38	DEROG15	Pricing derogation	Numeric	70, 100
39	DEROG16	Pricing derogation	Nominal	OP, RE, SA, ...
40	TAILLE1	Risk size	Ordinal	01 - [0 -250k], 02 - [250k-50
41	TAILLE2	Risk size	Ordinal	01 - [0 -250k], 02 - [250k-50
42	CA1	Revenue data	Numeric	0, 750, 2250, 4000, ...
43	CA2	Revenue data	Numeric	0, 750, 2250, 4000, ...
44	CA3	Revenue data	Numeric	0, 750, 2250, 4000, ...
45	KAPITAL1	Capital data	Binary Flag	0, 1
53	KAPITAL9	Capital data	Binary Flag	0, 1

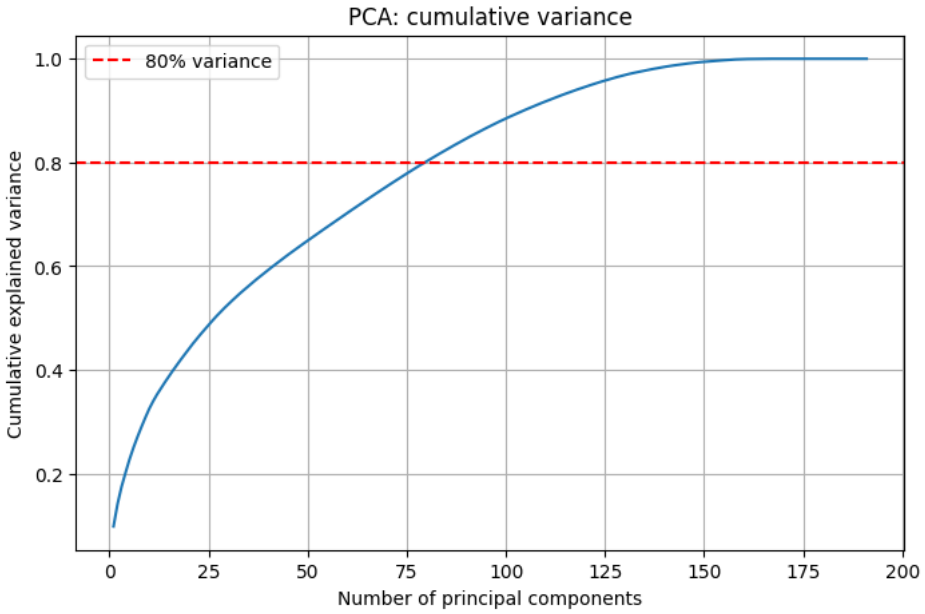
1.2 Missingness Analysis





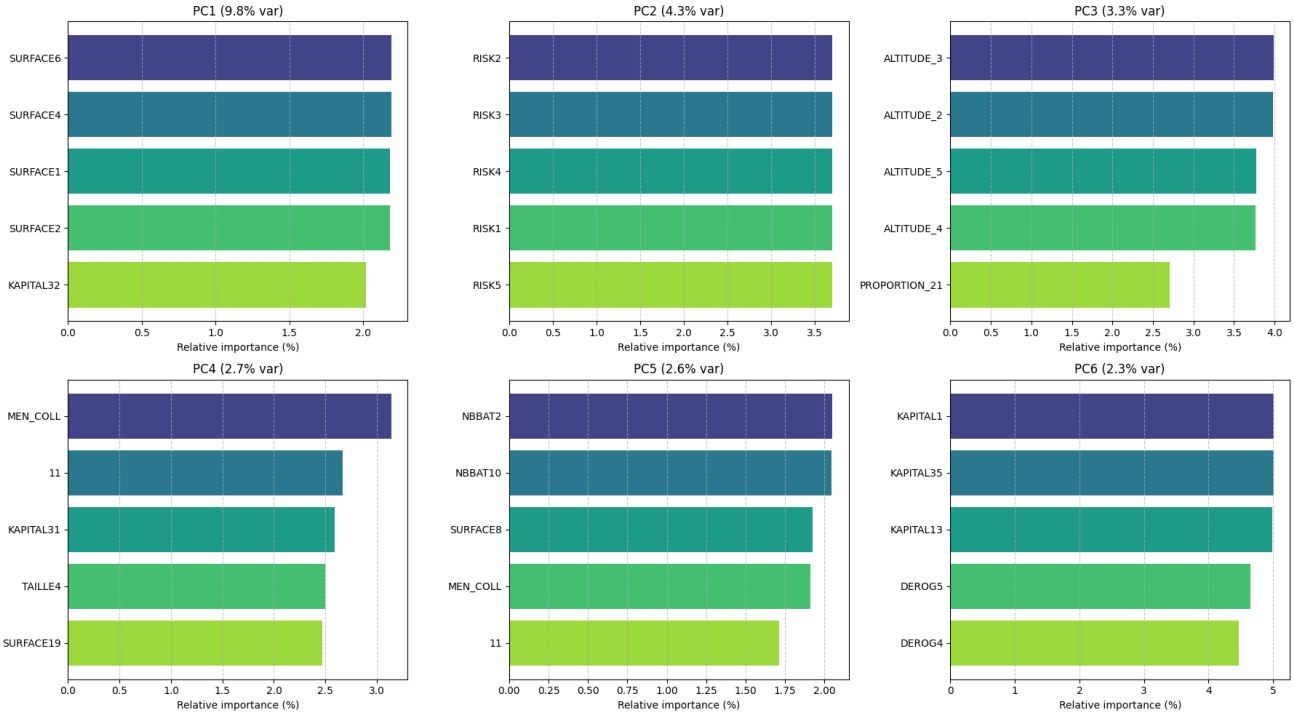

```
Columns missing >50%: 183
Mean pairwise missingness correlation: 0.923
High correlation confirms a single systematic block (MNAR), not random dropout.
```

2.2 PCA

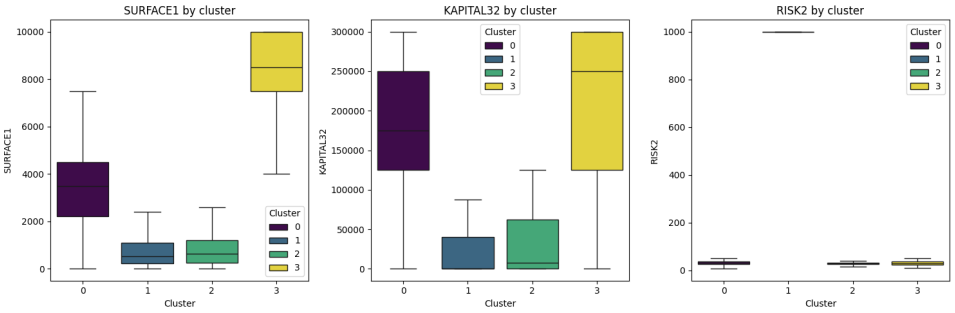


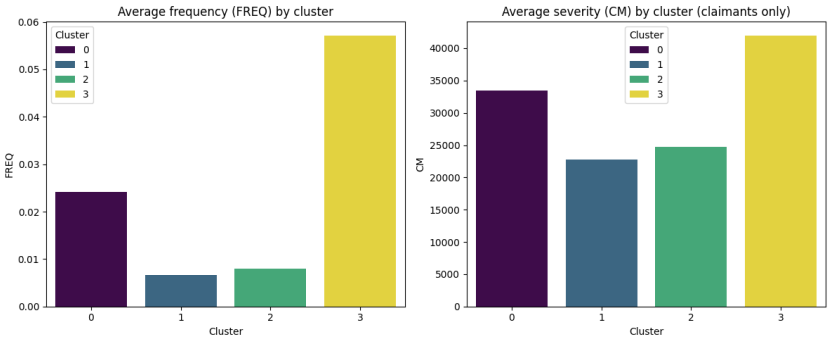
Principal components summary:

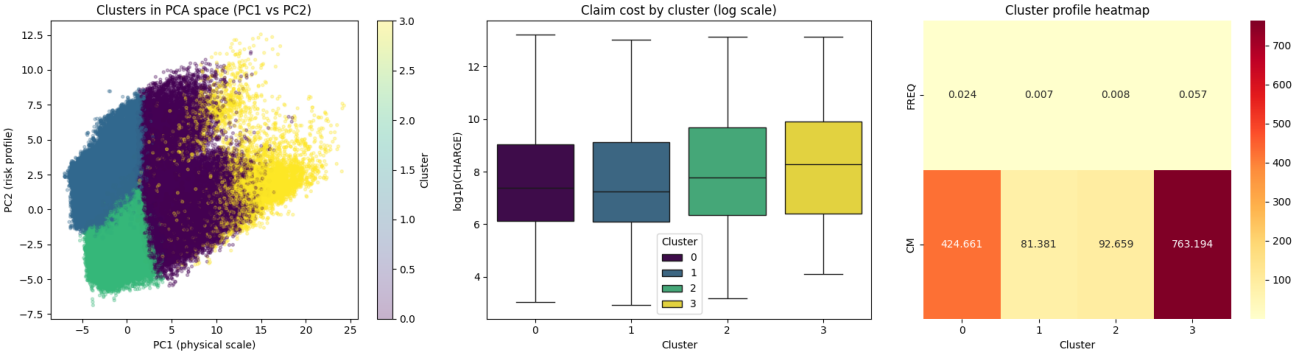
	Variance (%)	Top 1	Top 2	Top 3
PC1	9.84%	(2.2%) SURFACE6	(2.2%) SURFACE4	(2.2%) SURFACE1
PC2	4.27%	(3.7%) RISK2	(3.7%) RISK3	(3.7%) RISK4
PC3	3.32%	(4.0%) ALTITUDE_3	(4.0%) ALTITUDE_2	(3.8%) ALTITUDE_5
PC4	2.69%	(3.1%) MEN_COLL	(2.7%) 11	(2.6%) KAPITAL31
PC5	2.57%	(2.1%) NBBAT2	(2.0%) NBBAT10	(1.9%) SURFACE8
PC6	2.27%	(5.0%) KAPITAL1	(5.0%) KAPITAL35	(5.0%) KAPITAL13



2.3 Clustering

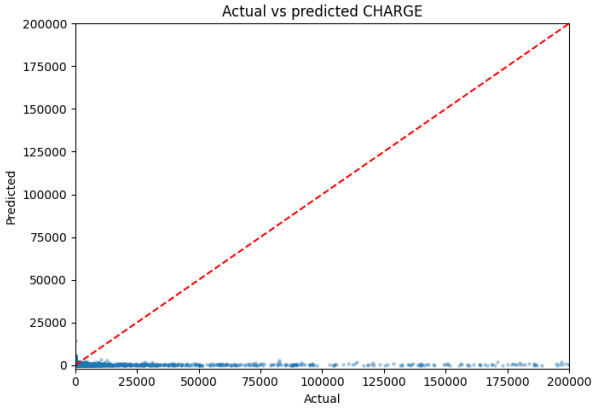


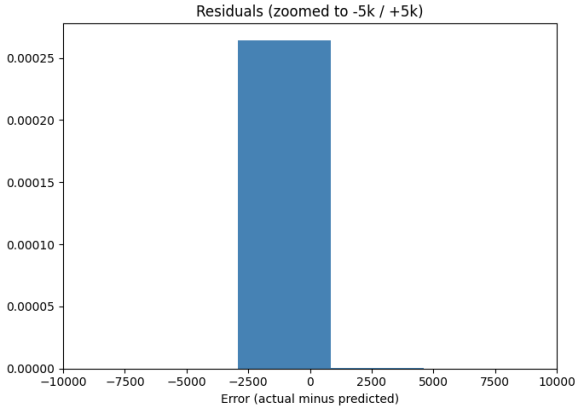




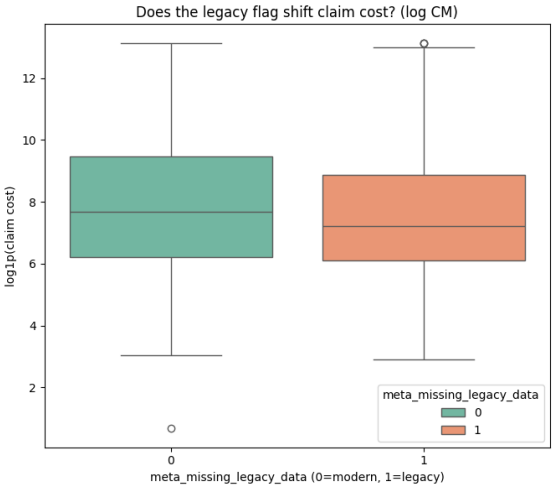
3.4 Evaluation

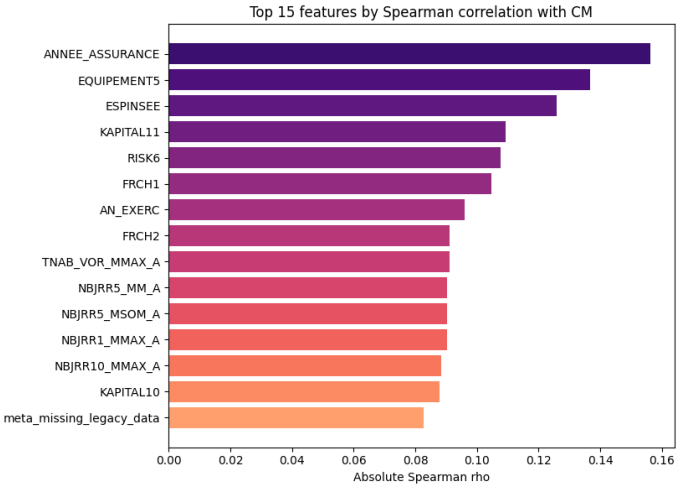
```
sub-model metrics:
FREQ  RMSE: 0.3570 | MAE: 0.0220 | MedAE: 0.0052
CM    RMSE: 79865.49 | MAE: 34637.54 | MedAE: 13881.63

final CHARGE:
RMSE: 6796.20 | MAE: 306.20 | MedAE: 46.52
```





3.5 Does the legacy flag shift claim costs?





4.2 Model Explainability